

8. Introduction: Why This Matters

Welcome back, everyone. If you have been following along with our coding journey, you've likely hit a wall with SQL aggregation. It is one of the most common stumbling blocks for developers transitioning into data roles or backend engineering. You know how to use `SELECT *`, and you know that `SUM()` gives you a total. But what happens when you need a total for every single row?

This is where the magic—and the confusion—happens. Imagine you are analyzing employee salaries. You want to know each person's salary, their department, and specifically what percentage of their *own* department's total salary they earn. If you simply use `SUM(salary) OVER ()`, you get a global average. That isn't useful if your goal is relative comparison within a group.

Interviewers love testing this specific concept because it demonstrates an engineer's maturity. Junior developers often collapse data using `GROUP BY` when the requirement actually demands row-level detail preserved alongside aggregate context. By mastering the `PARTITION BY` clause, you unlock the ability to perform complex comparative analysis without losing the granularity of your original dataset. This isn't just about syntax; it's about understanding *context* in data processing. In this lesson, we will demystify how window functions act as a "reset button" for calculations, allowing us to compare an individual record against the properties of its group simultaneously.

9. Problem Overview

Let's rephrase the core challenge we are tackling today. We have a dataset containing information about employees (or students), including their `name`, `department` (or class), and `salary` (or test score).

The goal is to enrich this dataset with new columns that represent aggregate statistics calculated *within* specific sub-groups, rather than globally across the entire table.

Specifically, we need to solve two main requirements: 1. **Department Total:** For every single employee row, calculate the sum of salaries for their specific department. 2. **Department Context Ratio:** Determine how that employee's salary compares to that same department total (e.g., calculating a percentage).

Crucially, we must **not** collapse the data. We cannot simply say "Department A has a total of \$1M" and stop there. We need every row to appear, but each row needs to know about its peer group's aggregate.

This is distinct from a standard `GROUP BY` query, which would merge all rows belonging to Department A into one summary line.

10. Examples

To make this concrete, let's look at a dataset of employees in two departments: Engineering and Marketing.

Input Data

Name	Department	Salary
Alice	Engineering	90,000
Bob	Engineering	120,000
Charlie	Engineering	80,000
Diana	Marketing	60,000
Eve	Marketing	45,000

Desired Output

We want a result set that keeps all five rows but adds a `Department Total` and an `Individual Ratio`.

Name	Department	Salary	Department Total	Individual Ratio
Alice	Engineering	90,000	290,000	31.0%
Bob	Engineering	120,000	290,000	41.3%
Charlie	Engineering	80,000	290,000	27.5%
Diana	Marketing	60,000	105,000	57.1%
Eve	Marketing	45,000	105,000	42.8%

Notice how the `Department Total` repeats for Alice and Bob because they belong to the same group. The calculation resets when it moves from Engineering to Marketing.

11. Intuition: The "Pocket Wallet" Analogy

How does an experienced engineer discover the solution? It starts with visualization. Imagine you have a wallet divided into distinct pockets, where each pocket represents a department.

Inside each pocket are several bills (the salaries). If you ask, "What is the total money in my wallet?" that's a global sum. But if you ask, "How much money do I have in my *left* pocket?" you need to look only at that specific area.

Now, imagine trying to tell this computer to count the bills in each pocket while keeping all the bills visible on the table. You cannot just "sum up" and move to the next line, because that would discard the individual bills. You need a mechanism that says: "Sum everything in Pocket A, write the result down, then immediately start summing from zero for Pocket B."

This is exactly what `PARTITION BY` does. It acts as the mental boundary between groups. Without it, the SQL engine treats the entire table as one giant pocket. With `PARTITION BY Department`, you tell the engine: "Stop counting when the department name changes."

Think of a classroom analogy mentioned in our video. If you calculate the class average without resetting for each section (e.g., Period 1 vs. Period 2), you mix apples and oranges. `PARTITION BY` is that moment where the teacher says, "Okay, we start fresh for this specific group." It allows us to compare an individual record against the properties of the group they belong to simultaneously.

12. Brute Force Solution

Before we use the elegant window functions, let's imagine the "naive" approach: using `JOIN` or subqueries to calculate totals first.

The Idea

1. Run a query to get the total salary for *every* department without grouping individuals: `sql`
`SELECT department, SUM(salary) as dept_total FROM employees GROUP BY department;`
2. Join this result back to the original table on the `department` column.

Algorithm

```
# Pseudocode Logic for Brute Force
total_table = GROUP_BY(employees, ['department']).sum('salary')
result = JOIN(employees, total_table, 'on employees.department == total_table.department')
```

Advantages

- **Understandable:** If you haven't learned window functions yet, this is logical.
- **Familiar:** Uses standard `GROUP BY` and `JOIN`, which everyone knows.

Disadvantages

- **Inefficient ($O(N \times M)$):** We iterate over the table twice. First to get totals, then to join. This can be slow on massive datasets.
- **Messy Syntax:** Requires multiple steps or complex subqueries in a single statement (`SELECT ... FROM ... WHERE EXISTS...`).
- **Hard to Extend:** Adding a third metric, like a running total within the department, becomes exponentially harder with joins.

Complexity

- **Time:** $O(N \times M)$ where N is rows and M is groups (though often optimized by DB engines, logically it's two passes).
- **Space:** Requires storing intermediate results for the grouped totals.

13. Optimal Solution: Window Functions with `PARTITION BY`

The optimal way to solve this is using a **Window Function**. A window function performs an aggregation over a set of rows (the window) related to the current row, without collapsing them.

The Core Syntax

```
SELECT
    name,
    department,
    salary,
    SUM(salary) OVER (PARTITION BY department) AS dept_total
FROM employees;
```

Step-by-Step Breakdown

1. **`OVER (...)`** : This keyword tells SQL that we are using a window function, not an aggregate collapse.
2. **`SUM(salary)`** : The aggregation logic. We want the sum of salaries.
3. **`PARTITION BY department`** : This is the magic key. It defines the "window." The engine calculates the sum separately for every distinct value in the `department` column.
 - For rows where `department` = 'Engineering', the window includes Alice, Bob, and Charlie. `SUM` returns 290,000 for all three.
 - When the row switches to `department` = 'Marketing', the window resets. It now only looks at Diana and Eve. `SUM` returns 105,000.

This allows us to see the forest (the department total) while standing in a specific tree (our individual salary row).

Diagram: How the Window Resets

Row Sequence	Current Dept	Window Frame (Rows Included for SUM)	Result

1	Engineering	[Alice]	--
2	Engineering	[Alice, Bob]	--
3	Engineering	[Alice, Bob, Charlie]	290,000 (for all 3)
4	Marketing	[Diana, Eve]	105,000 (for both)

(Note: In window functions, the frame includes the current row and all previous rows in the partition by default for cumulative calculations, but for a simple SUM of the whole group, it effectively sums everyone in that partition).

14. Python Code (SQL Logic in Python Context)

While we are discussing SQL, let's simulate this logic using Python and Pandas to ensure you understand the underlying mechanics before running it in a database. This is often helpful for data engineers who need to preprocess data locally.

```
import pandas as pd

# Create Sample DataFrame
data = {
    'name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
    'department': ['Engineering', 'Engineering', 'Engineering',
                  'Marketing', 'Marketing'],
    'salary': [90000, 120000, 80000, 60000, 45000]
}
df = pd.DataFrame(data)

# The Python Equivalent of SQL PARTITION BY
# groupby creates the partitions, apply performs the calculation per partition
# transform broadcasts the result back to every row in that partition

# Calculate Department Total and reset index for clarity
dept_total = df.groupby('department')['salary'].transform('sum')

# Calculate Ratio (Individual Salary / Department Total)
df['individual_ratio'] = df['salary'] / dept_total

# Display Result
print(df[['name', 'department', 'salary', 'individual_ratio']].round(2))
```

15. Code Walkthrough

Let's analyze the Python solution line by line to demystify what is happening under the hood, which mirrors the SQL logic perfectly.

- `df.groupby('department')` : This creates a dictionary-like object where keys are departments and values are DataFrames containing rows for that department. Think of this as separating the "wallet" into pockets.
- `['salary']` : We select the specific column to aggregate (the bills inside the pocket).
- `.transform('sum')` : This is the critical method. Unlike `agg`, which returns a single number per group, `transform` applies the function (Sum) and then expands the result back to the original shape of the DataFrame. If there are 3 rows in Engineering, `transform` outputs the total sum 3 times.
- **Broadcasting**: The resulting series (`dept_total`) has a length matching the original DataFrame. When we divide `df['salary']` by this new column, Pandas aligns them by index position. Since the values in `dept_total` are identical for rows within the same group, the division works perfectly to create the ratio for each individual.

16. Dry Run: Walking Through One Example

Let's manually trace the logic for Alice (Engineering) and Diana (Marketing).

Initial State: * Row 1: Alice, Eng, \$90k * Row 4: Diana, Marketing, \$60k

Step 1: Grouping The engine identifies partitions. * Partition A: {Alice, Bob, Charlie} (Total = \$290k) * Partition B: {Diana, Eve} (Total = \$105k)

Step 2: Calculation The `SUM` function runs within the partition boundaries defined by `PARTITION BY`. * For Alice: The window looks at {Alice, Bob, Charlie}. Sum = 290k. * For Diana: The window looks at {Diana, Eve}. Sum = 105k.

Step 3: Assignment The calculated total is assigned to every row in that partition. * Alice gets \$290k. * Bob gets \$290k. * Diana gets \$105k. * Eve gets \$105k.

Step 4: Final Output Generation The query returns the original columns plus the new `dept_total` column, maintaining the original row count. No rows are deleted or merged.

17. Complexity Analysis

Why is this solution optimal?

- **Time Complexity:** $O(N)$.
 - The database engine scans the table once (or twice internally with an index, but logically one pass per partition). It processes each row exactly once to determine its group and adds the pre-calculated aggregate. This is linear time.
 - Comparing this to the brute force join ($O(N \times M)$), window functions are significantly faster on large datasets because they avoid materializing intermediate joined tables.
- **Space Complexity:** $O(N)$.
 - The result set has the same number of rows as the input, plus one or two new columns for the aggregates. We do not create a separate massive temporary table to hold group summaries that are then joined back; we calculate on the fly.

18. Alternative Solutions

You might ask: "Can I use `CROSS JOIN` with a subquery?"

The Subquery Approach

```
SELECT
    e.name,
    e.department,
    e.salary,
    s.total_salary
FROM employees e
CROSS JOIN (
    SELECT department, SUM(salary) as total_salary
    FROM employees
    GROUP BY department
) s
WHERE e.department = s.department;
```

Comparison

- **Readability:** Window functions are cleaner. The subquery requires a `WHERE` clause to filter the join result back down, which can get messy with multiple columns.
- **Performance:** On modern databases (PostgreSQL, MySQL 8+, SQL Server), window functions utilize specific execution plans that are highly optimized and often faster than nested joins for this specific pattern.

- **Scalability:** If you need a running total *within* the department (e.g., cumulative salary by hire date), window functions (`SUM()` `OVER` (`ORDER BY ... PARTITION BY ...`)) handle this natively. Using `GROUP BY` and `CROSS JOIN` becomes incredibly difficult to extend.

19. Edge Cases

Robust SQL code must handle edge cases gracefully. Here is what happens:

1. **Empty Input:** If the `employees` table is empty, the query returns zero rows. No errors occur because the window simply has no partitions to process.
2. **Single Row in Department:** If a department has only one employee (e.g., the CEO alone), `SUM(salary)` equals that employee's salary. The ratio becomes \$1.0\$ (or 100%). This is mathematically correct and expected.
3. **NULL Values:** If an employee's salary is `NULL`, `SUM` ignores it in the total calculation, but `NULL / Total` results in `NULL` for that row. You may need to handle this with `COALESCE` if you want to display 0 or a specific message.
4. **Negative Numbers:** If salaries (or test scores) can be negative, the sum still works correctly. The ratio remains valid as long as the total is not zero.
5. **Zero Department Total:** If every salary in a department is \$0\$, dividing by the total causes a division-by-zero error in some SQL dialects. Always check for `WHERE total > 0` if calculating percentages.

20. Common Mistakes

Based on years of reviewing interview code, here are five specific pitfalls to avoid:

1. **Forgetting `PARTITION BY`** : Writing `SUM(salary) OVER ()` calculates the global total for *everyone*. This destroys the context you need.
2. **Using `GROUP BY` instead:** This collapses rows into a summary line, losing the individual names and salaries entirely.
3. **Missing `OVER` Clause:** Simply writing `SUM(salary) FROM employees WHERE department = 'Eng'` returns a single scalar value, not a column you can join back to the original rows easily.
- 4.

Quick Review

When to use PARTITION BY with Window Functions?

Use when comparing a row's value to its group's aggregate metrics without collapsing the result set.

What is the time complexity of standard window functions like SUM or AVG?

$O(n)$ per partition, as they must scan and aggregate all rows within that specific group once.

What is the space complexity for maintaining a running total across a partition?

$O(k)$ where k is the size of the current partition, needed to store intermediate cumulative values.

What is the most common bug when using PARTITION BY?

Forgetting to include PARTITION BY, causing aggregates to calculate over the entire table instead of specific groups.

How does adding ORDER BY change a window function like SUM?

It calculates a cumulative sum (running total) sequentially within each partition rather than a simple group total.

Follow-up: How do you exclude the current row from a 'same group' comparison?

Use LAG or LEAD to shift values, allowing comparison of the current row against its immediate neighbor in the partition.

sql Lesson 19: PARTITION BY with Window Functions (PARTITION BY divides your result set into distinct sub-groups over which window functions perform calculations without collapsing individual rows into a single summary line. It enables complex comparative analysis by allowing you to keep row-level detail while simultaneously accessing aggregate metrics from the surrounding context. You reach for this whenever you need to compare an individual record against the properties of the group to which it belongs, such as calculating percentages of a total or identifying relative rankings.) — free Python cheat sheet